

Software Requirement Specifications

AeroVision: Drone Detection with Machine Learning



Submitted by

Rimsha Malik (S23BINFT1M01062)

Submitted to

Dr. Syed Ali Nawaz Shah

Department of Information Technology

Faculty of Computing

The Islamia University of Bahawalpur

Meeting Details

Sr No	Details	Date	Supervisor Signature
1.	Project Idea discussion (online)	19-03-2026	
2.	Project Proposal Acceptance (online)	23-03-2026	
3.	SRS Discussion	22-04-2026	

Summary

This is the Software Requirements Specification document for AeroVision, a real-time drone detection system based on machine learning. The project goal is deceptively simple: given an image, is there a drone or not? We live in a time where drones are everywhere and people have found many ways to make use of them illegally. To this end, AeroVision trains a YOLOv8 deep learning model on one of the benchmark datasets for drone detection available on Kaggle and wraps it inside a neat Streamlit web app so that anyone can upload an image and get instant results. This document thus includes the project overview and scope, its users, competing and related systems, functional and non-functional requirements, and key use case decomposition.

Table of Contents

1. Introduction	1
1.1. Purpose	1
1.2. Scope	2

1.3.	Product Perspective	2
1.4.	User Characteristics	2
1.5.	Similar apps and systems/Literature Review	3
1.6.	Proposed Technologies	3
2.	Requirements	4
2.1.	Function Requirements	4
2.1.1.	Image Upload.....	4
2.1.2.	Drone Detection.....	5
2.1.3.	Result Display.....	5
2.1.4.	No Drone Notification	6
2.1.5.	File Validation	6
2.2.	Non-Functional Requirements	7
3.	Use Cases and Flow of Processes	8
3.1.	UC001	9
3.2.	UC002	11
3.3.	UC003	13
3.4.	UC004	15
3.5.	UC005	16
4.	References	18

1. Introduction

Up in the air, drones now show up more often than before. From snapping photos to helping farms, guiding rescue teams during emergencies, handling missions in war zones - they do quite a lot. One handles deliveries too, gliding through city skies with parcels tucked underneath. Yet their growing numbers stir unease about who might be watching, what could go wrong, how systems stay protected. A small machine slipping past guards, drifting above secure sites like army grounds or runways, maybe hovering over someone's backyard unseen. Older security setups fail when spotting tiny, quick drones. Because they weren't made for such tasks. AeroVision steps in right there - tackling that exact gap. Not just another tool, but one shaped by machine learning. It works like this: feed it a picture, then watch it decide if a drone hides inside. If yes, a frame appears, outlining where it sees the object. Alongside comes a number showing its certainty level. Training happens on open data anyone can check - no hidden sources. Then everything wraps into a clean web interface using Streamlit. Easy reach, even for those who avoid tech talk.

1.1. Purpose

What AeroVision does is teach computers to spot drones in pictures without needing people to watch every second. Instead of counting on slow, costly human monitoring filled with mistakes, it uses smart algorithms to stay alert. A key part of this effort focuses on shaping a YOLOv8 model to recognize flying devices clearly and correctly. Testing happens against known image sets so performance can be measured honestly. Once trained, findings show up through a basic website - no special skills needed to understand them. Behind the scenes, the idea proves newer neural methods fit tough safety jobs in open skies.

1.2. Scope

From your browser, you can upload pictures so AeroVision spots drones inside them. After sending an image, the tool runs it through a tuned YOLOv8 setup that marks drone locations with boxes plus shows how sure it is about each one. This build uses Kaggle to train the model on a standard set of drone photos, comes with code for finding drones, and features a Streamlit-powered website interface. However, watching real-time video isn't part of this release; neither is spotting objects besides drones, supporting phones, signing users in, storing data, or running on remote servers. Those ideas stay saved for later versions, well past what's ready now.

1.3. Product Perspective

AeroVision runs on its own, right there on your desktop. Not tied to anything else, yet built so it can grow later. Inside, two pieces connect - one part learns how drones appear, using best.pt, a smart file shaped by training and a Streamlit frontend that lets users interact with that model without writing any code. Think of it like this: the model is the expert, and the web app is the interface that lets you talk to the expert. In the future, this same model could be plugged into a CCTV system, a drone management platform, or a government surveillance network.

1.4. User Characteristics

General Users (Primary Users):

- Basic knowledge of using web applications
- No technical or machine learning expertise required
- Upload images and view detection results

Security Personnel (Operational Users):

- Basic to moderate technical knowledge
- Use the system to detect drones in surveillance images
- Make decisions based on detection results for safety and monitoring purposes

1.5. Similar apps and systems/Literature Review

Back in 2019, some researchers put together the DroneDetect collection - one of the early big sets of audio and camera footage meant for spotting drones. That project leaned heavily on radio wave tracking rather than recognizing shapes in images. Before AeroVision showed up, tools already existed. Take DJI's own AeroScope system - it gets used at airfields and by officials to catch drones through their signals. Expensive? Yes. Then came a study titled "Real-Time UAV Detection and Anti-UAV System," published in 2020. It tested YOLO-style models on live video, found them pretty good at finding flying devices. Trouble was, those setups needed powerful graphics processors just to keep up. Some models for spotting drones live on Roboflow Universe, built by hobbyists using YOLOv5 or YOLOv8 - open to everyone yet tricky for non-tech users. What sets AeroVision apart comes down to simplicity: it pairs a strong YOLOv8 setup with a browser-based front door, no downloads needed. Instead of wrestling with tools behind the scenes, people just open a page and go.

1.6. Proposed Technologies

AeroVision is built entirely in Python. Ultralytics YOLOv8 is the best way to detect objects in real time right now. It doesn't need much code to train and deploy. The training takes place on Kaggle Notebooks, which give you free access to GPUs and are great for deep learning experiments because you don't need to buy expensive hardware. We use Streamlit for the web interface because it lets us make interactive apps in Python without needing HTML, CSS, or JavaScript. OpenCV takes care of all the image processing, such as reading images and drawing bounding boxes. Streamlit works best with PIL images, so Pillow is used to change the format of images. NumPy handles array operations on image data under the hood. The dataset comes from Kaggle's publicly available drone detection collection in YOLO format, which means the annotations are already prepared and ready for training.

2. Requirements

The core feature of the system is the detection of drones in an image uploaded by the user. To accomplish this, the system must upload images, accept a variety of image formats, process the model for inference, and display the results clearly and simply enough for a first-time machine learning user. On the modeling side, the system must be properly trained with sufficient examples of labeled drones to learn how to generalize to unseen images. Training involves splitting the data into training and validation subsets, setting up the model, running many epochs, and saving the best-performing weights. After training and downloading the best weights, the Streamlit app loads the model once at startup and uses it for all subsequent detections, keeping response times low.

2.1. Function Requirements

The following are the functional requirements that describe what the system must do from the user's perspective.

2.1.1. Image Upload

- **Name:** FR001
- **Purpose:** The user must be able to upload an image file so the system can analyze it for drone presence.
- **User(s):** General User
- **Input:**
 - *Image File:* A JPG, JPEG, or PNG image selected from the user's device.
 - *File Size:* Must be under 10MB to ensure smooth processing.
 - *Image Format:* Only image files are accepted. Non-image files will be rejected.
- **Output:** The uploaded image is previewed on screen and passed to the detection pipeline for processing.

2.1.2. Drone Detection

- **Name:** FR002
- **Purpose:** The system must run the trained YOLOv8 model on the uploaded image and identify whether any drone is present.
- **User(s):** General User
- **Input:**
 - *Uploaded Image:* The validated image file received from the upload step.
 - *Model File (best.pt):* The trained YOLOv8 weights loaded at application startup.
 - *Confidence Threshold:* A minimum confidence value (default 0.5) below which detections are ignored.
- **Output:** The model returns bounding box coordinates and confidence scores for every drone detected in the image. If no drone is found, an empty result set is returned.

2.1.3. Result Display

- **Name:** FR003
- **Purpose:** The system must clearly show the detection results to the user in a way that is easy to understand, even without a technical background.
- **User(s):** General User
- **Input:**
 - *Annotated Image:* The output image with bounding boxes drawn around detected drones by OpenCV.
 - *Detection Data:* The list of confidence scores and drone count returned by the model.
- **Output:** The system displays the original image and the annotated output image side by side, along with the total number of drones detected and the confidence score for each one.

2.1.4. No Drone Notification

- **Name:** FR004
- **Purpose:** When the model finds no drone in the uploaded image, the system must inform the user clearly rather than showing a blank or confusing result.
- **User(s):** General User
- **Input:**
 - *Detection Result:* An empty result set returned by the model indicating zero drones were found.
- **Output:** The system displays the message "*No drones detected in this image.*" along with the original uploaded image so the user knows the analysis was completed successfully.

2.1.5. File Validation

3. **Name:** FR005
4. **Purpose:** The system must check every uploaded file before sending it to the model, to prevent errors caused by unsupported or corrupted files.
5. **User(s):** General User
6. **Input:**
 - 6.1. *Uploaded File:* Any file submitted by the user through the upload button.
 - 6.2. *File Extension:* The format of the file (e.g., .pdf, .txt, .exe, .jpg).
 - 6.3. *File Size:* The size of the file in megabytes.
7. **Output:** If the file is a valid image (JPG, JPEG, or PNG) and under 10MB, it is accepted and passed to the upload step. If the file is invalid, the system displays the error message "*Unsupported file format. Please upload a JPG, JPEG, or PNG image.*" and prompts the user to try again.

2.2 Non-Functional Requirements

Performance: The detection result must be returned within 5 seconds of the image being uploaded, assuming standard laptop hardware without a dedicated GPU.

Usability: The interface must be clean and simple enough that a user with no machine learning background can complete a detection task within 30 seconds of opening the app without any instructions.

Accuracy: The trained model must achieve a minimum mAP (mean Average Precision) of 70% on the validation set of the benchmark dataset.

Reliability: The system must handle at least 50 consecutive image uploads in a single session without crashing or slowing down.

Portability: The application must run on any machine with Python 3.9 or higher installed, on Windows, macOS, or Linux, without requiring a GPU.

Maintainability: The codebase must be organized into clearly separated modules (detect.py and streamlit_app.py) so that the detection logic can be updated independently of the user interface.

3. Use Cases and Flow of Processes

The use cases below describe the key interactions between the user and AeroVision. The primary actor in all cases is the General User, who accesses the system through the Streamlit web interface running locally on their machine. The system itself acts as a secondary actor by loading the model and processing the image.

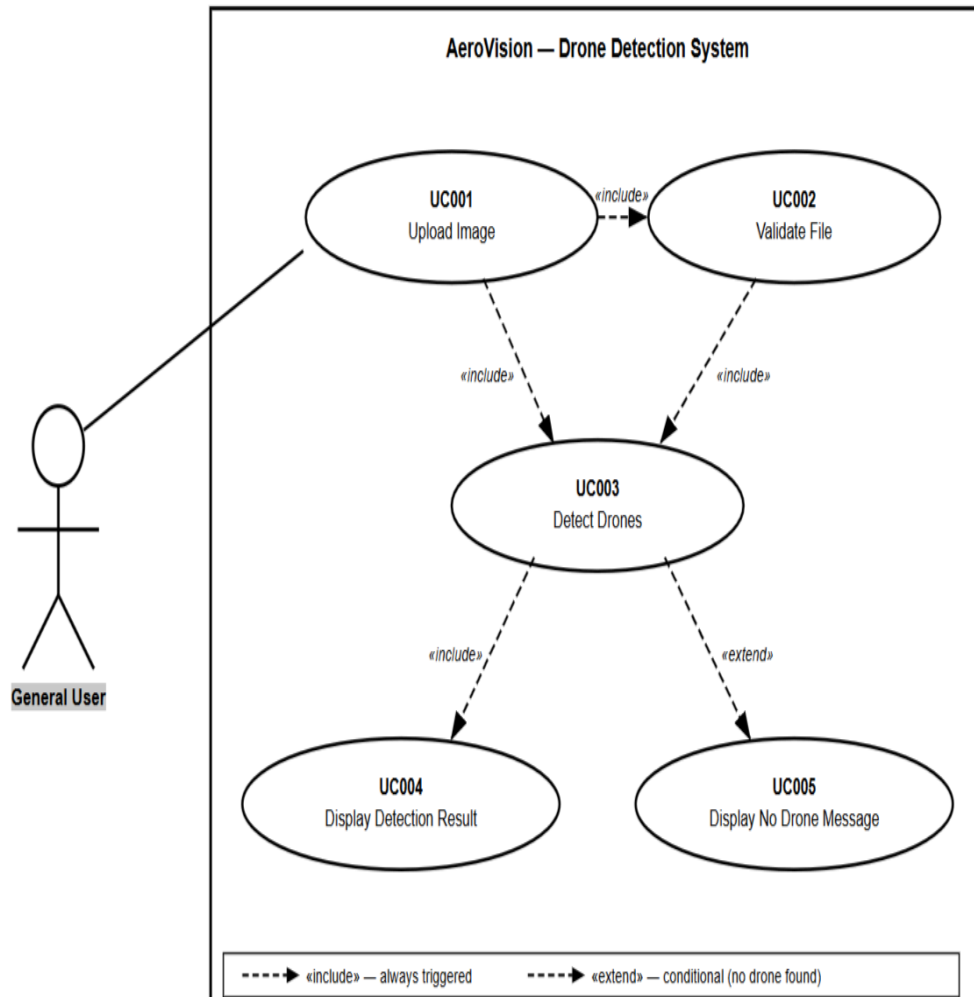


Figure 3.1: System Level Use Case Diagram

3.1 UC001 — Upload Image

ID	UC001
Name	Upload Image
Description	The user uploads an image file for drone detection through the AeroVision web interface.
Requirement(s)	FR001
Actor(s)	General User
Precondition	The AeroVision Streamlit app is running and accessible in the browser.
Postcondition	The uploaded image is previewed on screen and passed to the file validation process.
Basic Flow	1. Basic Flow 1. User opens the AeroVision app in the browser. 2. User clicks the "Upload Image" button. 3. User selects a JPG, JPEG, or PNG file from their device. 4. System previews the image on screen. 5. System passes the image to UC002 for validation. Alternative Flow : 1. User drags and drops an image directly onto the upload area. 2. System accepts it the same way as a manually selected file. Exceptions 1. User closes the file dialog without selecting anything — system stays on the upload screen with no change.

Diagram:

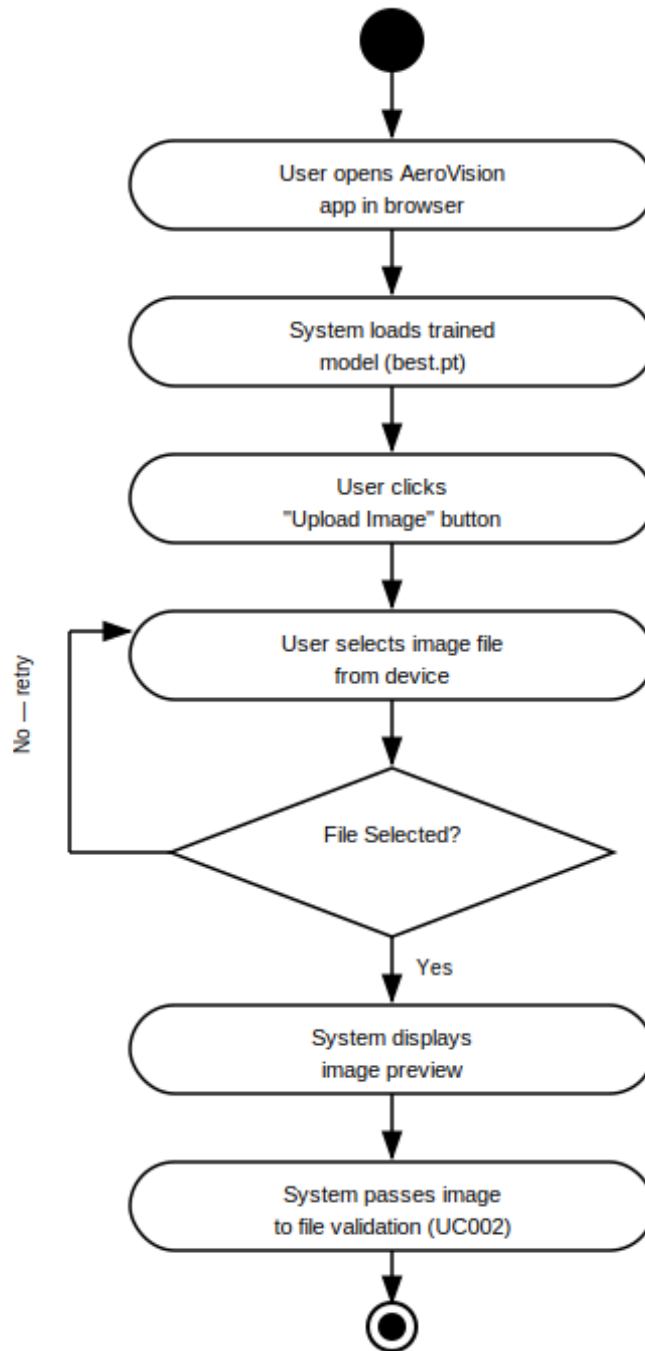


Figure 3.2 Activity Diagram UC001 : Upload Image

3.2 UC002 — Validate File

ID	UC002
Name	Validate File
Description	This use case describes how the system checks the uploaded file to make sure it is a supported image format and within the allowed size limit before passing it to the detection model.
Requirement(s)	FR005
Actor(s)	General User
Precondition	A file has been uploaded by the user through UC001.
Postcondition	If valid, the image is forwarded to UC003 for detection. If invalid, an error message is shown and the user is asked to upload again.
Basic Flow	<i>Basic Flow</i> 1. System reads the file extension of the uploaded file. 2. System checks that the extension is JPG, JPEG, or PNG. 3. System checks that the file size is under 10MB. 4. All checks pass — system forwards the image to UC003. <i>Alternative Flow</i> 1. File extension is valid but image is slightly over size limit. 2. System displays a warning and asks the user to compress or resize the image. <i>Exceptions</i> 1. File is not an image (e.g., PDF, TXT, EXE) — system displays: <i>"Unsupported file format. Please upload a JPG, JPEG, or PNG image."</i> 2. File is corrupted and cannot be read — system displays: <i>"Unable to read the file. Please try a different image."</i>

Diagram:

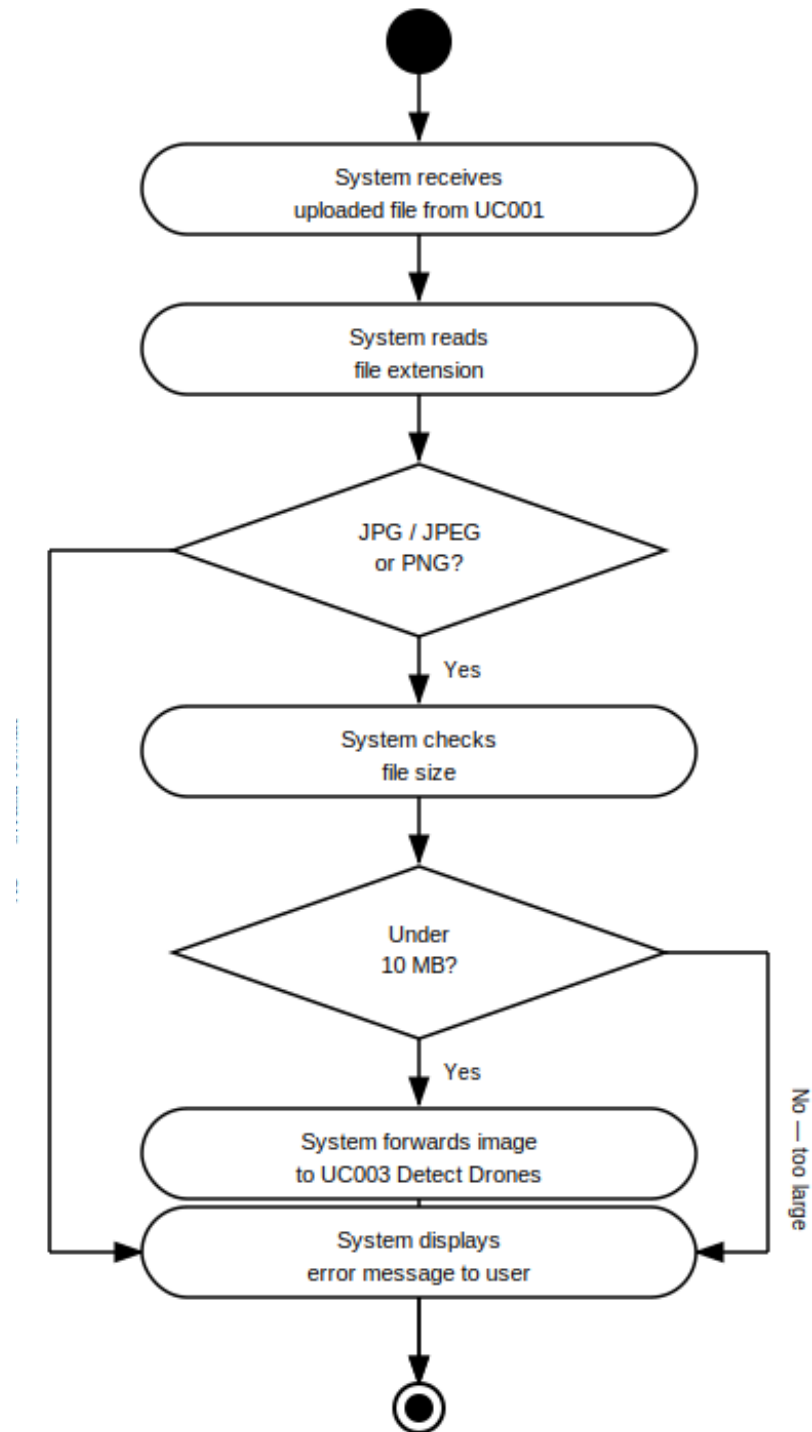


Figure 3.3 Activity Diagram UC002 : Validate File

3.3 UC003 — Detect Drones

ID	UC003
Name	Detect Drones
Description	This use case describes how the system passes the validated image through the trained YOLOv8 model to identify whether any drones are present and where they are located in the image.
Requirement(s)	FR002
Actor(s)	General User
Precondition	The image has passed file validation in UC002. The trained model file (best.pt) is loaded in memory at application startup.
Postcondition	The model returns either a list of bounding boxes with confidence scores, or an empty result. The system proceeds to either UC004 or UC005 depending on the result.
Basic Flow	<i>Basic Flow</i> 1. System receives the validated image from UC002. 2. System passes the image to detect.py. 3. detect.py loads best.pt and runs YOLOv8 inference on the image. 4. Model returns bounding box coordinates and confidence scores for all detected drones. 5. System checks if the result is non-empty. 6. System forwards the result to UC004 for display. <i>Alternative Flow</i> 1. Image is valid but very low resolution or heavily blurred. 2. Model still runs inference but returns low confidence scores. 3. System proceeds to UC004 and displays results with a note about low image quality. <i>Exceptions</i>

	1. Model file (best.pt) is missing from the model/ directory — system displays: <i>"Model not found. Please place best.pt in the model/ folder."</i> 2. Detection takes longer than 10 seconds — system displays a timeout warning.
--	---

Diagram:

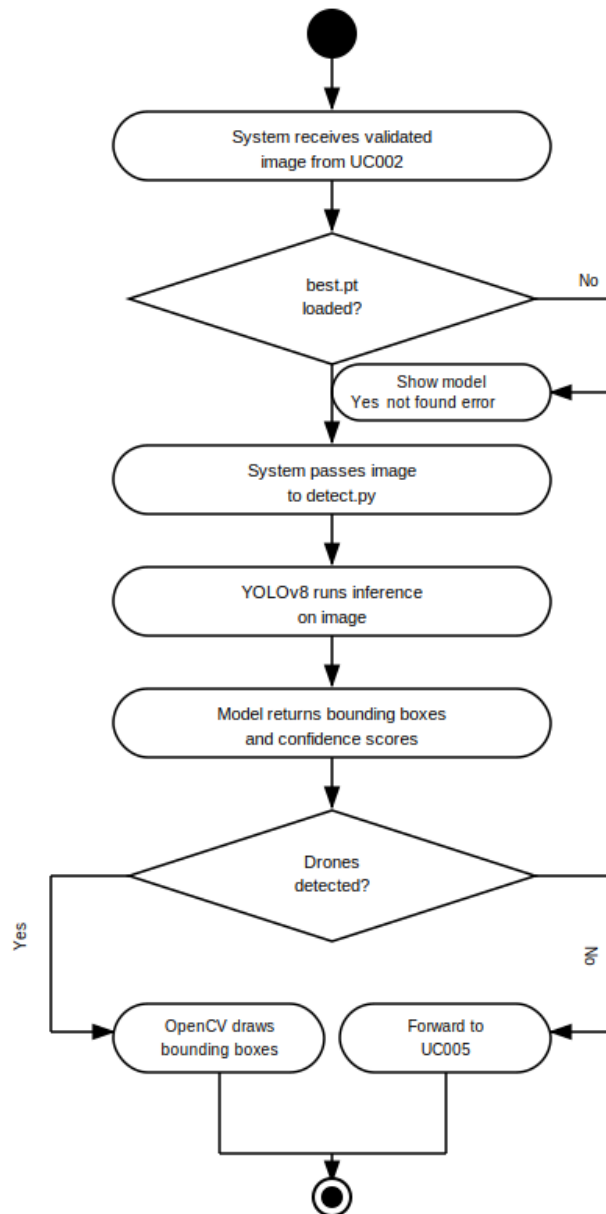


Figure 3.3 Activity Diagram UC003 : Detect Drones

3.4 UC004 — Display Detection Result

ID	UC004
Name	Display Detection Result
Description	This use case describes how the system presents the drone detection results to the user in a clear and readable format, including the annotated image and detection statistics.
Requirement(s)	FR003
Actor(s)	General User
Precondition	UC003 has completed and returned at least one drone detection with bounding box coordinates and a confidence score.
Postcondition	The user sees the annotated output image with bounding boxes drawn around detected drones, along with the total drone count and individual confidence scores.
Basic Flow	<i>Basic Flow</i> 1. System receives the bounding box coordinates and confidence scores from UC003. 2. OpenCV draws bounding boxes around each detected drone on the image. 3. Confidence score is printed as a label on each bounding box. 4. System displays the original uploaded image and the annotated result image side by side. 5. System shows the total number of drones detected and lists each confidence score below the images. 6. User reads the results. <i>Alternative Flow</i> 1. Multiple drones are detected in the same image. 2. System draws a separate bounding box for each drone and lists all confidence scores individually.

	Exceptions 1. OpenCV fails to draw bounding boxes due to an unexpected image format issue — system displays the raw detection data as text instead.
--	--

3.5 UC005 — Display No Drone Message

ID	UC005
Name	Display No Drone Message
Description	This use case describes what happens when the detection model finds no drone in the uploaded image. Instead of leaving the user confused with an empty screen, the system clearly communicates that the analysis was completed but nothing was found.
Requirement(s)	FR004
Actor(s)	General User
Precondition	UC003 has completed and returned an empty result, meaning no drones were detected in the image.
Postcondition	The user sees a clear message confirming that no drones were found, along with the original image so they can confirm the right image was analyzed.
Basic Flow	Basic Flow 1. System receives an empty detection result from UC003. 2. System does not attempt to draw any bounding boxes. 3. System displays the original uploaded image as-is. 4. System displays the message: <i>"No drones detected in this image."</i> 5. User reads the message and understands the analysis is complete. 6. User can choose to upload a different image. Alternative Flow

	1. User is unsure whether the model missed a drone or there genuinely is none. 2. User re-uploads the same image — system runs detection again and returns the same result.
--	--

Diagram:

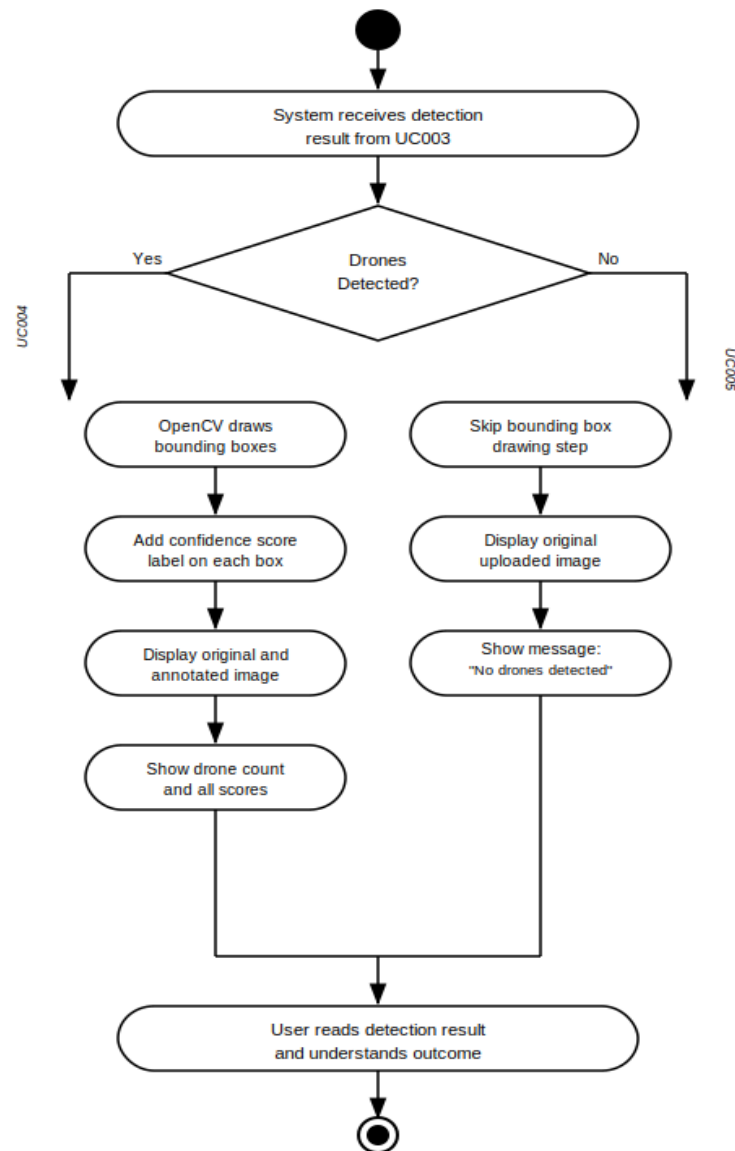


Figure 3.4 Activity Diagram UC004 & UC005 : Result Display

4. References

- [1] J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," *arXiv preprint arXiv:1804.02767*, 2018.
- [2] G. Jocher et al., "Ultralytics YOLOv8," *GitHub repository*, 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [3] A. Singla, "Drone Detection Dataset," *Kaggle*, 2022. [Online]. Available: <https://www.kaggle.com/datasets/cybersimar08/drone-detection/data>
- [4] Streamlit Inc., "Streamlit Documentation," 2023. [Online]. Available: <https://docs.streamlit.io>
- [5] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.
- [6] R. Srivastav, "Real-Time UAV Detection and Anti-UAV System," *IEEE Transactions on Intelligent Transportation Systems*, vol. 22, no. 8, pp. 5209-5218, 2021.